Exoplanet Atmospheric Spectral Analysis Pipeline

Table of Contents

1. Overview
2. Installation & Requirements
3. Quick Start Guide
4. API Reference
5. Scientific Background
6. Data Processing Workflow
7. Troubleshooting
8. Examples
9. Performance Optimization
10. References

Overview

The Exoplanet Atmospheric Spectral Analysis Pipeline is a comprehensive Python toolkit designed to process raw telescope spectroscopic data and detect atmospheric molecular signatures in exoplanet spectra. The pipeline implements state-of-the-art algorithms for data cleaning, noise reduction, and molecular detection specifically optimized for transit spectroscopy and direct imaging observations.

Key Features

- **Advanced data cleaning**: Cosmic ray removal, instrumental correction, telluric subtraction
- **Molecular detection**: Template-based cross-correlation for 5+ molecular species
- **Quantitative analysis**: Abundance fitting with statistical significance testing
- **Visualization**: Comprehensive plotting and reporting capabilities
- **Synthetic data generation**: Built-in test data generator for validation

Supported Molecules

- **H_2O** (Water vapor): Primary biosignature, greenhouse gas
- **CO_2** (Carbon dioxide): Atmospheric composition indicator
- **CH_4** (Methane): Potential biosignature, atmospheric chemistry
- **O_2** (Oxygen): Primary biosignature for photosynthesis

- **O_3** (Ozone): Secondary biosignature from O_2 photochemistry

---

Installation & Requirements

Dependencies

```bash
pip install numpy scipy matplotlib scikit-learn
```

Required Packages

- **numpy** ≥ 1.20.0: Array operations and numerical computing
- **scipy** ≥ 1.7.0: Signal processing and optimization
- **matplotlib** ≥ 3.3.0: Plotting and visualization
- **scikit-learn** ≥ 0.24.0: Data preprocessing and PCA

Optional Dependencies

```bash
pip install astropy photutils specutils  # For FITS file handling
```

Quick Start Guide

Basic Usage

```python
from exoplanet_pipeline import ExoplanetSpectralPipeline
import numpy as np

# Initialize pipeline
pipeline = ExoplanetSpectralPipeline()

# Load your data (wavelength in nm, flux normalized)
wavelength = np.loadtxt('wavelength.txt')  # Your wavelength array
raw_spectrum = np.loadtxt('spectrum.txt')  # Your flux array

# Process the spectrum
results = pipeline.process_spectrum(raw_spectrum, wavelength)

# View detected molecules
print("Detected molecules:", results['detected_molecules'])
```

Using Synthetic Data

```python
from exoplanet_pipeline import generate_synthetic_exoplanet_data, ExoplanetSpectralPipeline

# Generate test data
wavelength, raw_spectrum = generate_synthetic_exoplanet_data()

# Process
pipeline = ExoplanetSpectralPipeline()
results = pipeline.process_spectrum(raw_spectrum, wavelength)
```

API Reference

Class: `TelescopeDataCleaner`

Primary class for cleaning raw telescope spectroscopic data.

Methods

`remove_cosmic_rays(spectrum, threshold=5)`

Removes cosmic ray hits using median filtering and statistical outlier detection.

Parameters:

- `spectrum` (array): Input flux array
- `threshold` (float): Detection threshold in median absolute deviations

Returns:

- `clean_spectrum` (array): Spectrum with cosmic rays removed
- `cosmic_mask` (array): Boolean mask of cosmic ray locations

Example:

```python
cleaner = TelescopeDataCleaner()
clean_spec, cosmic_mask = cleaner.remove_cosmic_rays(raw_spectrum, threshold=5)
print(f"Removed {np.sum(cosmic_mask)} cosmic ray hits")
```

`correct_instrumental_response(spectrum, wavelength)`

Corrects for wavelength-dependent detector efficiency and non-linearity.

Parameters:

- `spectrum` (array): Input flux array
- `wavelength` (array): Wavelength array in nm

Returns:

- `corrected_spectrum` (array): Instrumentally corrected spectrum

`remove_telluric_absorption(spectrum, wavelength, telluric_model=None)`

Removes Earth's atmospheric absorption features.

Parameters:

- `spectrum` (array): Input flux array
- `wavelength` (array): Wavelength array in nm
- `telluric_model` (array, optional): Custom telluric absorption model

Returns:

- `corrected_spectrum` (array): Telluric-corrected spectrum
- `telluric_model` (array): Applied telluric model

`denoise_spectrum(spectrum, method='savgol', **kwargs)`

Applies various denoising algorithms.

Parameters:

- `spectrum` (array): Input flux array
- `method` (str): Denoising method ('savgol', 'gaussian', 'median')
- `**kwargs`: Method-specific parameters

Savitzky-Golay Parameters:

- `window_length` (int): Filter window length (default: 15)
- `polyorder` (int): Polynomial order (default: 3)

Example:

```python
# Savitzky-Golay smoothing
denoised = cleaner.denoise_spectrum(spectrum, method='savgol',
                                    window_length=21, polyorder=3)

# Gaussian smoothing
denoised = cleaner.denoise_spectrum(spectrum, method='gaussian', sigma=2.0)
```

Class: `AtmosphericSignatureDetector`

Detects and analyzes molecular atmospheric signatures.

Methods

`generate_molecular_template(molecule, wavelength_grid)`

Generates absorption template for specified molecule.

Parameters:

- `molecule` (str): Molecule name ('H2O', 'CO2', 'CH4', 'O2', 'O3')
- `wavelength_grid` (array): Wavelength array for template generation

Returns:

- `template` (array): Normalized absorption template

`cross_correlate_molecules(spectrum, wavelength, molecules=None)`

Cross-correlates spectrum with molecular templates.

Parameters:

- `spectrum` (array): Input spectrum
- `wavelength` (array): Wavelength array
- `molecules` (list, optional): List of molecules to test

Returns:

- `results` (dict): Detection results for each molecule

Result Structure:

```python
{
    'H2O': {
        'correlation': array,          # Cross-correlation function
        'max_correlation': float,      # Peak correlation value
        'significance': float,         # Detection significance (σ)
        'detected': bool               # Above threshold detection
    }
}
```

`fit_molecular_abundances(spectrum, wavelength, molecules)`

Fits molecular abundances using least-squares optimization.

Parameters:

- `spectrum` (array): Input spectrum
- `wavelength` (array): Wavelength array
- `molecules` (list): List of molecules to fit

Returns:

- `fit_results` (dict): Fitting results and statistics

Class: `ExoplanetSpectralPipeline`

Complete end-to-end processing pipeline.

Methods

`process_spectrum(raw_spectrum, wavelength, plot_results=True)`

Executes complete processing pipeline.

Parameters:

- `raw_spectrum` (array): Raw flux measurements
- `wavelength` (array): Wavelength array in nm
- `plot_results` (bool): Generate diagnostic plots

Returns:

- `results` (dict): Complete analysis results

Results Structure:

```python
{
    'processed_spectrum': array,      # Final cleaned spectrum
    'wavelength': array,              # Wavelength grid
    'cosmic_rays': array,             # Cosmic ray mask
    'telluric_model': array,          # Telluric correction model
    'correlations': dict,             # Molecular correlations
    'abundance_fit': dict,            # Abundance fitting results
    'detected_molecules': list        # List of detected molecules
}
```

Scientific Background

Transit Spectroscopy

Transit spectroscopy observes exoplanets as they pass in front of their host stars. The planet's atmosphere absorbs specific wavelengths of starlight, creating characteristic absorption features that reveal atmospheric composition.

Key Concepts:

- **Transit depth**: Fraction of starlight blocked by planet
- **Atmospheric scale height**: Vertical extent of detectable atmosphere
- **Molecular absorption**: Wavelength-specific opacity from molecules

Molecular Signatures

Water Vapor (H_2O)

- **Primary bands**: 940, 1130, 1400, 1870 nm
- **Significance**: Essential for habitability, universal solvent
- **Detection**: Strong absorption features, easily detectable

Carbon Dioxide (CO_2)

- **Primary bands**: 1430, 1600, 2010 nm
- **Significance**: Greenhouse gas, atmospheric evolution indicator
- **Detection**: Moderate strength, requires good S/N

Methane (CH_4)

- **Primary bands**: 1660, 2300 nm
- **Significance**: Potential biosignature (biological/geological)
- **Detection**: Weak features, challenging detection

Oxygen (O_2)

- **Primary bands**: 760, 1270 nm
- **Significance**: Primary biosignature, photosynthesis product
- **Detection**: Very challenging due to telluric contamination

Ozone (O_3)

- **Primary bands**: 600, 1000 nm
- **Significance**: Secondary biosignature, O_2 photoproduct
- **Detection**: Broad absorption features

Statistical Significance

Detection significance is calculated as:

$$\sigma = (S - \mu) / \sigma_n$$

Where:

- S = Signal strength (correlation peak)

- μ = Mean noise level
- σ_n = Noise standard deviation

Detection Thresholds:

- **3σ**: Tentative detection (99.7% confidence)
- **5σ**: Strong detection (99.99997% confidence)

Data Processing Workflow

1. Data Ingestion

```python
# Load raw data
wavelength = np.loadtxt('wavelength.dat')    # nm
raw_flux = np.loadtxt('raw_spectrum.dat')    # counts or normalized
error = np.loadtxt('error_spectrum.dat')     # optional
```

2. Quality Assessment

```python
# Check data quality
print(f"Wavelength range: {wavelength.min():.1f} - {wavelength.max():.1f} nm")
print(f"Spectral resolution: {len(wavelength)} points")
print(f"Mean S/N ratio: {np.mean(raw_flux/error):.1f}")
```

3. Data Cleaning Pipeline

```python
cleaner = TelescopeDataCleaner()

# Step 1: Remove cosmic rays
spectrum, cosmic_mask = cleaner.remove_cosmic_rays(raw_flux, threshold=5)

# Step 2: Correct instrumental response
spectrum = cleaner.correct_instrumental_response(spectrum, wavelength)

# Step 3: Remove telluric absorption
spectrum, telluric = cleaner.remove_telluric_absorption(spectrum, wavelength)

# Step 4: Apply denoising
spectrum = cleaner.denoise_spectrum(spectrum, method='savgol')
```

4. Molecular Detection

```python
detector = AtmosphericSignatureDetector()

# Cross-correlation analysis
correlations = detector.cross_correlate_molecules(spectrum, wavelength)

# Identify detections
detected = [mol for mol, data in correlations.items() if data['detected']]
print(f"Detected molecules: {detected}")
```

5. Abundance Quantification

```python
if detected:
    # Fit molecular abundances
    fit_results = detector.fit_molecular_abundances(spectrum, wavelength, detected)

    # Extract abundances
    abundances = fit_results['abundances']
    for molecule, abundance in abundances.items():
        print(f"{molecule}: {abundance:.2e} (relative)")
```

6. Results Validation

```python
# Check fit quality
chi_squared = fit_results['chi_squared']
if chi_squared < 2.0:
    print("Good fit quality")
elif chi_squared < 5.0:
    print("Acceptable fit quality")
else:
    print("Poor fit quality - check data")
```

Troubleshooting

Common Issues

1. No Molecular Detections

Symptoms: All significance values < 3σ

Possible Causes:

- Low signal-to-noise ratio
- Insufficient spectral resolution
- Atmospheric features too weak
- Incorrect wavelength calibration

Solutions:

```python
# Reduce detection threshold (use with caution)
detector.detection_threshold = 2.0

# Apply stronger denoising
spectrum = cleaner.denoise_spectrum(spectrum, method='savgol',
                                    window_length=25, polyorder=3)

# Check wavelength calibration
plt.plot(wavelength, spectrum)
plt.xlabel('Wavelength (nm)')
plt.title('Check for systematic wavelength shifts')
```

2. Poor Fit Quality (High χ^2)

Symptoms: `chi_squared` > 5.0

Possible Causes:

- Systematic errors not removed
- Incorrect molecular templates
- Non-linear instrumental effects

Solutions:

```python
# Apply additional denoising
spectrum = cleaner.denoise_spectrum(spectrum, method='gaussian', sigma=1.5)

# Check for remaining systematics
residuals = fit_results['residuals']
plt.plot(wavelength, residuals)
plt.title('Check residuals for systematics')
```

3. False Positive Detections

Symptoms: Unrealistic molecular combinations

Possible Causes:

- Inadequate noise characterization
- Instrumental artifacts
- Telluric residuals

Solutions:

```python
# Increase detection threshold
detector.detection_threshold = 4.0

# Improve telluric correction
custom_telluric = load_telluric_model()  # Your telluric model
spectrum, _ = cleaner.remove_telluric_absorption(spectrum, wavelength,
                                                 custom_telluric)
```

Error Messages

LinAlgError: SVD did not converge

Cause: Singular matrix in abundance fitting **Solution**: Reduce number of molecules or improve data quality

ValueError: array lengths must match

Cause: Wavelength and spectrum arrays have different lengths **Solution**: Ensure consistent array dimensions

Performance Issues

Slow Processing

For large datasets (>10,000 points):

```python
# Downsample for initial analysis
n_sample = 5000
indices = np.linspace(0, len(wavelength)-1, n_sample, dtype=int)
wl_sample = wavelength[indices]
spec_sample = spectrum[indices]

# Process downsampled data
results = pipeline.process_spectrum(spec_sample, wl_sample)
```

Examples

Example 1: Basic Analysis

python

```python
import numpy as np
from exoplanet_pipeline import ExoplanetSpectralPipeline

# Load your data
wavelength = np.loadtxt('my_wavelength.txt')
spectrum = np.loadtxt('my_spectrum.txt')

# Initialize and run pipeline
pipeline = ExoplanetSpectralPipeline()
results = pipeline.process_spectrum(spectrum, wavelength)

# Print summary
print("Detected molecules:", results['detected_molecules'])
if results['abundance_fit']:
    for mol, abundance in results['abundance_fit']['abundances'].items():
        print(f"{mol}: {abundance:.2e}")
```

Example 2: Custom Molecular Templates

python

```python
# Extend detector with custom molecule
detector = AtmosphericSignatureDetector()

# Add ammonia (NH3) template
detector.molecular_templates['NH3'] = {
    'wavelengths': np.array([1500, 2000]),
    'depths': np.array([0.01, 0.015]),
    'widths': np.array([20, 25])
}

# Use in analysis
molecules = ['H2O', 'CO2', 'NH3']
correlations = detector.cross_correlate_molecules(spectrum, wavelength, molecules)
```

Example 3: Batch Processing

python

```python
import glob
import os

# Process multiple files
data_files = glob.glob('data/*.txt')
results_summary = {}

for file_path in data_files:
    filename = os.path.basename(file_path)

    # Load data (assuming wavelength in first column, flux in second)
    data = np.loadtxt(file_path)
    wavelength = data[:, 0]
    spectrum = data[:, 1]

    # Process
    pipeline = ExoplanetSpectralPipeline()
    results = pipeline.process_spectrum(spectrum, wavelength, plot_results=False)

    # Store results
    results_summary[filename] = {
        'detected_molecules': results['detected_molecules'],
        'n_detections': len(results['detected_molecules'])
    }

# Print summary
for filename, summary in results_summary.items():
    print(f"{filename}: {summary['n_detections']} detections - {summary['detected_molecules']}")
```

Example 4: Advanced Visualization

```python
import matplotlib.pyplot as plt

# Process spectrum
results = pipeline.process_spectrum(spectrum, wavelength)

# Create custom plot
fig, ax = plt.subplots(figsize=(12, 6))

# Plot spectrum with molecular regions highlighted
ax.plot(wavelength, results['processed_spectrum'], 'k-', alpha=0.8, label='Spectrum')

# Highlight detected molecular bands
for molecule in results['detected_molecules']:
    template_data = pipeline.detector.molecular_templates[molecule]
    for center in template_data['wavelengths']:
        if wavelength.min() <= center <= wavelength.max():
            ax.axvline(center, color='red', alpha=0.5, linestyle='--')
            ax.text(center, ax.get_ylim()[1]*0.9, molecule,
                    rotation=90, ha='center', va='top')

ax.set_xlabel('Wavelength (nm)')
ax.set_ylabel('Normalized Flux')
ax.set_title('Exoplanet Atmospheric Spectrum with Molecular Identifications')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Performance Optimization

Memory Usage

For large datasets, consider:

```python
# Process in chunks
def process_large_spectrum(wavelength, spectrum, chunk_size=5000):
    n_chunks = len(wavelength) // chunk_size + 1
    results = []

    for i in range(n_chunks):
        start_idx = i * chunk_size
        end_idx = min((i + 1) * chunk_size, len(wavelength))

        wl_chunk = wavelength[start_idx:end_idx]
        spec_chunk = spectrum[start_idx:end_idx]

        # Process chunk
        pipeline = ExoplanetSpectralPipeline()
        chunk_results = pipeline.process_spectrum(spec_chunk, wl_chunk,
                                                  plot_results=False)
        results.append(chunk_results)

    return results
```

Computational Speed

1. Optimize Cross-Correlation

```python
# Use scipy's optimized correlation
from scipy.signal import correlate

# Pre-compute normalized templates
normalized_templates = {}
for molecule in molecules:
    template = detector.generate_molecular_template(molecule, wavelength)
    normalized_templates[molecule] = (template - np.mean(template)) / np.std(template)
```

2. Parallel Processing

```python
from multiprocessing import Pool
import functools

def process_single_molecule(args):
    spectrum, wavelength, molecule = args
    detector = AtmosphericSignatureDetector()
    return detector.cross_correlate_molecules(spectrum, wavelength, [molecule])

# Parallel molecular detection
molecules = ['H2O', 'CO2', 'CH4', 'O2', 'O3']
args_list = [(spectrum, wavelength, mol) for mol in molecules]

with Pool() as pool:
    results = pool.map(process_single_molecule, args_list)
```

Memory-Efficient Large File Handling

```python
def process_fits_file(fits_filename):
    """Process large FITS files efficiently"""
    from astropy.io import fits

    with fits.open(fits_filename, memmap=True) as hdul:
        # Access data without loading entire file into memory
        wavelength = hdul[1].data['WAVELENGTH']
        spectrum = hdul[1].data['FLUX']

        # Process in chunks if needed
        if len(wavelength) > 50000:
            return process_large_spectrum(wavelength, spectrum)
        else:
            pipeline = ExoplanetSpectralPipeline()
            return pipeline.process_spectrum(spectrum, wavelength)
```

References

Scientific Papers

1. **Seager, S. & Sasselov, D. D.** (2000). "Theoretical Transmission Spectra during Extrasolar Giant Planet Transits." *Astrophysical Journal*, 537, 916-921.

2. **Brown, T. M.** (2001). "Transmission Spectra as Diagnostics of Extrasolar Giant Planet Atmospheres." *Astrophysical Journal*, 553, 1006-1026.
3. **Tinetti, G., et al.** (2007). "Water vapour in the atmosphere of a transiting extrasolar planet." *Nature*, 448, 169-171.
4. **Sing, D. K., et al.** (2016). "A continuum from clear to cloudy hot-Jupiter exoplanets without primordial water depletion." *Nature*, 529, 59-62.

Technical References

- **Cross-correlation technique**: Brogi, M., et al. (2012). *Astrophysical Journal*, 747, L16
- **Telluric correction**: Vacca, W. D., et al. (2003). *Publications of the Astronomical Society of the Pacific*, 115, 389
- **Instrumental calibration**: Coehlo, P., et al. (2005). *Astronomy & Astrophysics*, 443, 735

Software Documentation

- **NumPy**: Harris, C. R., et al. (2020). "Array programming with NumPy." *Nature*, 585, 357-362
- **SciPy**: Virtanen, P., et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python." *Nature Methods*, 17, 261-272

Online Resources

- **Exoplanet Archive**: https://exoplanetarchive.ipac.caltech.edu/
- **HITRAN Database**: https://hitran.org/ (molecular spectroscopy)
- **STScI Instruments**: https://www.stsci.edu/jwst/science-execution/approved-programs

---

Version History

v1.0.0 (Current)

- Initial release
- Basic molecular detection for H_2O, CO_2, CH_4, O_2, O_3
- Complete data cleaning pipeline
- Synthetic data generation
- Comprehensive documentation

Planned Features (v1.1.0)

- Additional molecules (NH_3, H_2S, SO_2)

- Bayesian parameter estimation
- Multi-planet batch processing
- FITS file support
- GUI interface

This documentation covers the complete functionality of the Exoplanet Atmospheric Spectral Analysis Pipeline. For additional support or feature requests, please refer to the project repository or contact the development team.